

BÁO CÁO THUYẾT MINH PHƯƠNG PHÁP TỐI ƯU HÓA MÔ HÌNH VÀ HỆ THỐNG AGENT ĐA TÁC VỤ

Đội thi: Bảng C - Innovator | Vietnamese Student HackAlthon 2026

Tóm tắt giải pháp (Abstract)

Báo cáo này thuyết minh giải pháp tối ưu hóa toàn diện cho mô hình ngôn ngữ lớn **Qwen3.5-4B-AWQ-4bit** nhằm xử lý hiệu quả bài toán trắc nghiệm đa ngành trong khuôn khổ Bảng C - Innovator. Giải pháp được thiết kế vận hành hoàn toàn offline trong môi trường ảo hóa Docker Container. Để khắc phục triệt để các hạn chế về suy luận logic (như lỗi truncation, lặp prompt, và nhận diện nhầm ký tự), nhóm tác giả đã tích hợp cơ chế Chain-of-Thought tối ưu hóa chiều dài, thuật toán Trích xuất trực tiếp (Direct Extraction), regex phân biệt chữ hoa/thường cùng hệ thống **Logit Margin Early Exit (Tối ưu hóa độ phân vân)**. Về mặt hệ thống, kỹ thuật Dynamic Pipelining trên vLLM LLMEngine cấp thấp đã được phát triển để tối đa hóa 100% công suất GPU. Kết quả thực nghiệm trên 463 câu hỏi Public Test đạt độ chính xác **88.98%** với tốc độ xử lý vượt trội đạt **1.42 giây/câu hỏi**.

1. Đặt vấn đề và Kiến trúc mô hình nền tảng

Trong khuôn khổ Bảng C - Innovator, thử thách đặt ra là phát triển một AI Agent có năng lực tự động xử lý và giải quyết các câu hỏi trắc nghiệm đa lĩnh vực có độ phức tạp cao (toán học, vật lý, luật học, kinh tế...). Yêu cầu kỹ thuật cốt lõi đòi hỏi AI Agent phải vận hành hoàn toàn **offline (không có kết nối mạng)** và được đóng gói trọn vẹn trong môi trường Docker Container. Điều này đặt ra hai bài toán kỹ thuật lớn: một là tối đa hóa độ chính xác của mô hình trên các tác vụ suy luận phức tạp, hai là tối ưu hóa thời gian thực thi (Throughput) trên tài nguyên GPU NVIDIA L4 (22GB VRAM) có hạn.

Nhóm tác giả đã lựa chọn mô hình nền tảng **Qwen3.5-4B-AWQ-4bit** để làm lõi xử lý. Mô hình đã được lượng tử hóa (quantized) giúp tiết kiệm đáng kể tài nguyên bộ nhớ đệm KV Cache và VRAM, cho phép vận hành với batch size lớn, đồng thời giữ vững năng lực lập luận ngữ cảnh xuất sắc từ kiến trúc gốc Qwen3.5. Để điều khiển mô hình ngoại tuyến, hệ thống giao tiếp trực tiếp với LLMEngine của thư viện vLLM.

2. Giải pháp tối ưu hóa thuật toán và kỹ thuật suy luận

Đối với các bài toán trắc nghiệm phức tạp, việc áp dụng suy luận zero-shot thông thường thường gặp phải những giới hạn về logic lập luận và bóc tách dữ liệu. Nhóm tác giả đã đề xuất và triển khai các cải tiến thuật toán quan trọng:

2.1. Lập luận Chain-of-Thought (CoT) thích ứng động

Các bài toán định lượng và lý luận luật học yêu cầu mô hình phải thực hiện nhiều bước biến đổi công thức và suy diễn trung gian. Nếu cấu hình giới hạn token sinh suy luận (`max_reasoning_tokens`) quá ngắn, luồng Chain-of-Thought sẽ bị cắt cụt (truncation) giữa chừng, khiến mô hình không thể hoàn tất bước suy luận cuối cùng để chốt đáp số. Nhóm tác giả đã thực hiện tối ưu hóa cấu hình và thiết lập mức giới hạn suy luận lên tới **800 tokens** cho Bước 1. Điều này đảm bảo mô hình có đủ không gian lập luận đầy đủ và chặt chẽ.

2.2. Cơ chế quyết định Logit Margin Early Exit (Dừng sớm theo độ tự tin)

Để tối ưu hóa thời gian thực thi mà không làm sụt giảm độ chính xác, nhóm tác giả đề xuất giải pháp **Logit Margin Early Exit**. Thay vì luôn chạy biểu quyết nhiều luồng cho tất cả các câu hỏi (tốn thời gian gấp 3 lần), hệ thống hoạt động như sau: 1. Chỉ chạy duy nhất **1 luồng CoT chính (Luồng 0)** để sinh suy luận chi tiết ở nhiệt độ thấp ($\text{temperature}=0.2$). 2. Sinh tiếp **1 token chốt đáp án** bằng giải thuật Greedy ($\text{temperature}=0.0$) kèm tham số $\text{logprobs}=5$ để đo phân phối xác suất của mô hình. 3. Tính toán **Logit Margin**:

$$\text{Margin} = P(\text{Lựa chọn 1}) - P(\text{Lựa chọn 2})$$

* Nếu $\text{Margin} \geq 0.50$ (Mô hình chọn dứt khoát, không phân vân): Lập tức thực hiện **Early Exit**, lấy đáp án Luồng 0 làm kết quả cuối cùng. Giải pháp này giúp tiết kiệm 66% thời gian tính toán cho các câu hỏi dễ. * Nếu $\text{Margin} < 0.50$ (Mô hình phân vân giữa các đáp án): Kích hoạt tiếp Luồng CoT 1 và Luồng CoT 2, chạy cơ chế **Majority Voting** (Biểu quyết đa số 3 luồng) để có đáp án chính xác nhất.

2.3. Bộ trích xuất ngôn ngữ phân biệt chữ hoa - thường (Vietnamese-Aware Pattern Matching)

Ngôn ngữ tự nhiên tiếng Việt chứa nhiều từ vựng có ký tự viết thường trùng với các nhãn đáp án trắc nghiệm (ví dụ: 'ngày', 'về', 'việc', 'hành'...). Nếu sử dụng bộ trích xuất regex không phân biệt chữ hoa/thường (case-insensitive), các ký tự viết thường này sẽ bị nhận diện sai thành đáp án. Để giải quyết triệt để, chúng tôi đã cấu hình bộ lọc biểu thức chính quy **phân biệt nghiêm ngặt chữ hoa/chữ thường (case-sensitive)** kết hợp với ranh giới từ (Word Boundary) dạng `\b[A-Z]\b` để định vị chính xác ký tự đáp án in hoa độc lập, loại bỏ hoàn toàn các ký tự trùng hợp trong tiếng Việt tự nhiên.

3. Giải pháp tối ưu hóa hệ thống và hiệu năng tính toán

Môi trường đánh giá Docker ngoại tuyến đặt ra thách thức về việc không được mở các cổng mạng HTTP hay socket. Do đó, kiến trúc hệ thống đã được tái cấu trúc chuyên sâu để đạt hiệu năng xử lý phần cứng cao nhất:

3.1. Cơ chế Gối đầu Bất đồng bộ Động (Dynamic Pipelining)

Để chạy offline hoàn toàn, giải pháp nạp trực tiếp mô hình vào GPU thông qua lớp đối tượng **LLMEngine** của vLLM. Nhằm duy trì công suất GPU luôn ở mức 100%, chúng tôi thiết lập vòng lặp sự kiện bất đồng bộ. Khi một câu hỏi hoàn thành Bước 1 và được phát hiện là bị cắt cụt (cần chạy thêm Bước 2), hệ thống lập tức gọi `engine.add_request()` để đẩy tiếp yêu cầu Bước 2 vào hàng đợi của vLLM ngay tại iteration tiếp theo. Nhờ đó, vLLM có thể batching gối đầu đồng thời Bước 1 của các câu sau với Bước 2 của các câu trước trên GPU, triệt tiêu hoàn toàn khoảng thời gian trễ chuyển tiếp.

3.2. Bỏ biên dịch FlashInfer JIT trong môi trường khép kín

Thư viện FlashInfer mặc định trong vLLM yêu cầu biên dịch runtime (JIT) bằng trình biên dịch CUDA (nvcc). Trong sandbox Docker ngoại tuyến của cuộc thi, việc thiếu trình biên dịch này sẽ gây lỗi crash hệ thống. Để khắc phục, giải pháp đã chuyển đổi backend Attention sang **FLASH_ATTN** đồng thời thiết lập biến môi trường `VLLM_USE_FLASHINFER_SAMPLER=0` trong Dockerfile. Sự điều chỉnh này đảm bảo container chạy cực kỳ ổn định và khởi động tức thì.

3.3. Prefix Caching và Quản lý VRAM tối ưu

Kích hoạt cấu hình `enable_prefix_caching=True` để vLLM chia sẻ dữ liệu bộ nhớ đệm KV Cache của prompt ngữ cảnh dùng chung giữa các luồng biểu quyết song song, giảm thiểu tài nguyên tính toán lặp lại. Đồng thời, cấu hình `gpu_memory_utilization=0.9` được tinh chỉnh để tận dụng tối đa 90% bộ nhớ VRAM của GPU NVIDIA L4 mà không gây lỗi Out-Of-Memory. Trước khi khởi động LLMEngine, script tự động quét và giải phóng các tiến trình nền GPU mồ côi (orphaned processes) trên máy chủ, đảm bảo GPU ở trạng thái sạch 100% tài nguyên trước khi vLLM phân bổ KV cache.

4. Thực nghiệm và Phân tích kết quả

Giải pháp được đánh giá thực nghiệm trực tiếp trên toàn bộ tập dữ liệu kiểm thử công khai (Public Test) gồm 463 câu hỏi.

4.1. Kết quả độ chính xác (Accuracy)

Giải pháp **Logit Margin Early Exit (Method 5)** đạt độ chính xác tổng thể ấn tượng là **88.98%** (đúng 412 / 463 câu hỏi), tăng vượt trội so với giải pháp chạy tuần tự thông thường và giải pháp biểu quyết cũ:

Phiên bản thử nghiệm	Số câu đúng / Tổng số	Độ chính xác (Accuracy)
Phiên bản baseline (Chưa tối ưu)	378 / 463	81.64%
Phiên bản Consensus Early Exit (Cũ)	406 / 463	87.69%
Phiên bản Logit Margin Early Exit (Đề xuất)	412 / 463	88.98% (+1.29% vs Consensus)

4.2. Kết quả tốc độ thực thi (Throughput)

Toàn bộ 463 câu hỏi trắc nghiệm hoàn thành suy luận trong **657.46 giây**. Thời gian xử lý trung bình đạt **1.42 giây / câu hỏi**. Nhờ cơ chế Early Exit, có tới **74.3% số câu hỏi để** kích hoạt dừng sớm chỉ sau Luồng 0, giúp tiết kiệm hơn 40% chi phí tính toán và đẩy Throughput hệ thống lên mức tối đa.

5. Kết luận

Phương pháp tiếp cận tối ưu bằng Logit Margin Early Exit của nhóm tác giả không chỉ nâng cao rõ rệt độ chính xác (+7.34% so với baseline và +1.29% so với Consensus) mà còn giảm thiểu chi phí suy luận của hệ thống vLLM trong môi trường Docker Container. Giải pháp này đạt độ tin cậy cực cao, hiệu năng vượt trội và hoàn toàn sẵn sàng cho vòng đánh giá Private Test tiếp theo.